

ОТЧЕТ ПО HW4

ФИО: Лебедев Андрей Андреевич

Группа: БПИ234

Email: Ananlebedev@edu.hse.ru

Цель работы — реализовать многопользовательское файловое хранилище MyDrive, работающее по протоколу TCP. Клиентское приложение должно синхронизировать заданную локальную директорию с сервером: отправлять серверу список локальных файлов, получать список отсутствующих или измененных файлов и передавать эти файлы по TCP. Дополнительно реализованы параллельная передача файлов через несколько TCP-соединений и режим передачи с использованием sendfile/DMA zero-copy.

Декларация об использовании ИИ

- *Я претендую на оценку до 10 баллов, генеративный ИИ не был использован для работы над этим заданием.*

Работа выполнена на языке: C++

Сетевой framework: Boost.Asio.

Coroutines не использовались.

Архитектура решения

Решение состоит из двух исполняемых файлов:

1. mydrive_server - серверное приложение.
2. mydrive_client - клиентское приложение.

Сервер запускается на удаленной Linux-машине и принимает TCP-подключения на порту 9000. Сервер использует Boost.Asio async accept loop и создает отдельную Session для каждого TCP-соединения. В рамках Session сервер читает фреймы

прикладного протокола через `async_read`, обрабатывает `manifest` клиента или принимает файл после сообщения `UPLOAD_BEGIN`.

Клиент запускается на моем ноутбуке с macOS. Клиент читает конфигурационный файл, сканирует директорию `client_files`, считает SHA-256 для каждого обычного файла и отправляет серверу `manifest`. После ответа сервера клиент создает до `N` TCP-соединений, где `N` задается параметром `max_connections`, и передает несколько файлов параллельно.

Синхронизация является односторонней: клиент -> сервер. Если файл добавлен только на сервере, клиент его не скачивает и не учитывает при следующем раунде синхронизации. Сервер сравнивает только те файлы, которые перечислены в `manifest` клиента.

Основные файлы проекта:

- `src/client/main.cpp` - клиент, CLI, сканирование директории, `manifest flow`, параллельные `upload workers`, `buffered/sendfile` передача.
- `src/server/main.cpp` - сервер, `async accept`, `Session`, прием `manifest` и файлов, хранение по `client_id`.
- `src/common/protocol.hpp` и `src/common/protocol.cpp` - бинарный протокол и сериализация/десериализация сообщений.
- `src/common/checksum.cpp` - потоковый расчет SHA-256.
- `src/common/config.cpp` - чтение JSON-конфигурации.

Экспериментальная установка

Клиент:

- ОС: macOS.
- Устройство: мой ноутбук.
- Локальный IP на скриншотах Wireshark: 192.168.1.8.
- Синхронизируемая директория: ./client_files.

Сервер:

- Удаленная Linux VM.
- IP-адрес сервера: 92.51.21.171.
- Порт сервера: 9000.
- Директория хранения на сервере: /root/mydrive-storage/./server_storage.

Конфигурация клиента:

```
{  
  "client_id": "lebedev-local",  
  "server_host": "92.51.21.171",  
  "server_port": 9000,  
  "directory": "./client_files",  
  "max_connections": 8,  
  "transfer_mode": "sendfile"  
}
```

Конфигурация сервера:

```
{  
  "listen_host": "0.0.0.0",  
  "listen_port": 9000,  
  "storage_root": "./server_storage",  
  "threads": 4  
}
```

Пользовательский интерфейс клиента

Клиент реализован как интерактивное консольное приложение. Доступные команды:

help - показать справку по командам.

sync - просканировать директорию и выполнить синхронизацию с сервером.

status - показать текущие настройки клиента и статистику последнего sync.

files - показать локальные файлы, их размеры и SHA-256.

mode buffered - включить обычную передачу через буфер в памяти.

mode sendfile - включить передачу через sendfile/DMA zero-copy.

connections N - задать число параллельных TCP-соединений, N от 1 до 32.

exit - завершить клиент.

```
> help  
Доступные команды:  
  help      показать эту справку  
  sync      просканировать директорию и синхронизировать файлы с сервером  
  status    показать текущие настройки клиента и статистику последней синхронизации  
  files     показать локальные файлы, их размеры и SHA-256  
  mode buffered  включить обычную передачу файла через буфер в памяти  
  mode sendfile  включить передачу через sendfile/DMA zero-copy  
  connections N  задать число параллельных TCP-соединений, N от 1 до 32  
  exit      завершить клиент
```

Бизнес-протокол

Для обмена служебными сообщениями реализован собственный бинарный протокол.

Каждое служебное сообщение передается как фрейм:

uint32 magic
uint16 version
uint16 message_type
uint32 payload_size
payload bytes

Получатель сначала читает фиксированный заголовок размером 12 байт, затем читает ровно `payload_size` байт и только после этого десериализует сообщение. Таким образом учитывается, что TCP является потоковым протоколом и не сохраняет границы сообщений.

Типы сообщений:

MANIFEST_REQUEST - клиент отправляет `client_id` и список файлов.

MANIFEST_RESPONSE - сервер отправляет список файлов, которые нужно загрузить.

UPLOAD_BEGIN - клиент сообщает имя, размер, SHA-256 и режим передачи файла.

UPLOAD_READY - сервер подтверждает готовность принять файл.

UPLOAD_FINISHED - сервер сообщает результат приема и проверки checksum.

ERROR - сообщение об ошибке.

Содержимое файла не сериализуется как объект. После `UPLOAD_BEGIN` и `UPLOAD_READY`

клиент передает по тому же TCP-соединению ровно указанное количество байт файла.

В режиме `sendfile` эти байты передаются через системный вызов `sendfile`.

Ключевые фрагменты реализации протокола

Фрейм протокола описан в `src/common/protocol.hpp`:

```
constexpr std::uint32_t kProtocolMagic = 0x4d594452; // MYDR
constexpr std::uint16_t kProtocolVersion = 1;
constexpr std::size_t kFrameHeaderSize = 12;
```

```
enum class MessageType : std::uint16_t {
    ManifestRequest = 1,
    ManifestResponse = 2,
    UploadBegin = 3,
    UploadReady = 4,
    UploadFinished = 5,
    Error = 6,
};
```

На стороне сервера чтение фрейма реализовано через `Boost.Asio async_read`:

```
boost::asio::async_read(socket_, boost::asio::buffer(header_), ...);
boost::asio::async_read(socket_, boost::asio::buffer(frame_.payload), ...);
```

После получения `UPLOAD_BEGIN` сервер читает тело файла как поток байт до тех пор, пока не будет принято ровно `upload.size` байт.

Передача файла на клиенте реализована в двух режимах:

1. `buffered` - чтение файла блоками и отправка через `boost::asio::write`.
2. `sendfile` - передача через системный вызов `sendfile` без копирования файла в пользовательский буфер.

Передававшиеся файлы

Для сравнительных замеров buffered/sendfile использовались файлы:

file_50mb.bin - 50 MB

file_100mb.bin - 100 MB

file_250mb.bin - 250 MB

Также была выполнена проверка передачи крупного файла:

file_1500mb.bin - 1500 MB

Для генерации файлов в проект добавлены bash-скрипты:

```
./generate_files.sh client_files
```

```
./generate_big_file.sh client_files
```

```
./generate_random.sh --n 10 --dir client_files
```

Скрипт generate_files.sh создает файлы 50 MB, 150 MB и 200 MB.

Скрипт generate_big_file.sh создает крупный файл file_1500mb.bin размером 1500 MB.

Скрипт generate_random.sh создает N файлов случайного размера от 105 MB до 205 MB. Например, команда ./generate_random.sh --n 10 --dir client_files создает 10 файлов, каждый из которых удовлетворяет требованию задания о размере не менее 100 MB.

Запуск сервера

Команда на удаленной Linux VM:

```
./build/mydrive_server --config config/server.json
```

Вывод:

```
^Croot@4998171-sx68553:~/mydrive-storage# ./build/mydrive_server --config config/server.json
MyDrive Server
Listening on 0.0.0.0:9000
Storage: /root/mydrive-storage/./server_storage
Threads: 4
```

Запуск клиента

Команда на локальном устройстве

```
./build/mydrive_client --config config/client.json
```

Вывод:

```
MAC@MacBook-Pro-2 HW4_Lebedev % ./build/mydrive_client --config config/client.json
MyDrive Client
Client ID: lebedev-local
Directory: /Users/MAC/Pentatonix/HSE/Course_3/Networks/HW4_Lebedev/./client_files
Server: 92.51.21.171:9000
Mode: sendfile
Connections: 8

Введите help, чтобы увидеть доступные команды.
> █
```

Демонстрация передачи крупного файла в режиме sendfile

Команда клиента:

```
sync
```

Вывод клиента:

```
> sync
Scanning directory...
Found 1 files, total 1.46 GB
Server requires 1 files, total 1.46 GB
Uploading with 1 connections, mode sendfile...
[upload] file_1500mb.bin 1.46 GB done 59.17 Mbps
Sync completed. Uploaded: 1.46 GB, elapsed: 213.61s
```


Демонстрация передачи крупного файла в режиме buffered

Команды клиента:

mode buffered

sync

Вывод клиента:

```
> mode buffered
Mode: buffered

> sync
Scanning directory...
Found 1 files, total 1.46 GB
Server requires 1 files, total 1.46 GB
Uploading with 1 connections, mode buffered...
[upload] file_1500mb.bin 1.46 GB done 55.95 Mbps
Sync completed. Uploaded: 1.46 GB, elapsed: 225.85s
```

Сравнение buffered и sendfile

Для сравнения режимов использовались файлы 50 MB, 100 MB и 250 MB. Передача выполнялась с 3 параллельными TCP-соединениями.

Режим buffered:

connections 3

sync

Вывод:

```
> connections 3
Connections: 3

> sync
Scanning directory...
Found 3 files, total 400.00 MB
Server requires 3 files, total 400.00 MB
Uploading with 3 connections, mode buffered...
[upload] file_50mb.bin 50.00 MB done 21.43 Mbps
[upload] file_150mb.bin 150.00 MB done 26.72 Mbps
[upload] file_200mb.bin 200.00 MB done 30.81 Mbps
Sync completed. Uploaded: 400.00 MB, elapsed: 54.82s
```

Режим sendfile:

mode sendfile

sync

Вывод:

```
> mode sendfile
Mode: sendfile

> sync
Scanning directory...
Found 3 files, total 400.00 MB
Server requires 3 files, total 400.00 MB
Uploading with 3 connections, mode sendfile...
[upload] file_50mb.bin 50.00 MB done 19.86 Mbps
[upload] file_150mb.bin 150.00 MB done 27.40 Mbps
[upload] file_200mb.bin 200.00 MB done 32.40 Mbps
Sync completed. Uploaded: 400.00 MB, elapsed: 52.13s
```

Таблица замеров

Размер файла	buffered throughput	sendfile throughput	connections
50 MB	21.43 Mbps	19.86 Mbps	3
150 MB	26.72 Mbps	27.40 Mbps	3
200 MB	30.81 Mbps	32.40 Mbps	3

Общее время передачи набора 400 MB:

buffered: 54.82 s

sendfile: 52.13 s

По результатам эксперимента режим sendfile оказался немного быстрее на данном наборе: 52.13 s против 54.82 s. При этом для файла 50 MB buffered оказался быстрее, а для файлов 150 MB и 200 MB быстрее оказался sendfile. Это показывает, что преимущество sendfile зависит от размера файла и текущих сетевых условий, но на суммарном наборе из 400 MB режим sendfile дал меньшее общее время передачи.

Доказательство параллельной передачи

Во время передачи трех файлов было выполнено:

```
MAC@MacBook-Pro-2 HW4_Lebedev % netstat -an | grep 9000
tcp4      0 131400 240.0.0.2.62500      92.51.21.171.9000    ESTABLISHED
tcp4      0 131400 240.0.0.2.62501      92.51.21.171.9000    ESTABLISHED
tcp4      0 131400 240.0.0.2.62499      92.51.21.171.9000    ESTABLISHED
tcp4      0 0 240.0.0.2.62487      92.51.21.171.9000    TIME_WAIT
```

Три соединения ESTABLISHED к 92.51.21.171:9000 подтверждают, что клиент передавал несколько файлов параллельно через несколько TCP-соединений.

Wireshark

No.	Time	Source	Destination	Protocol	Length	Info
3948	57.549887	192.168.1.8	92.51.21.171	TCP	78	52076 → 9000 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3727523882 TSecr=0 SACK_PERM
3967	57.611629	92.51.21.171	192.168.1.8	TCP	54	9000 → 52076 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
5412	89.208550	192.168.1.8	92.51.21.171	TCP	78	54492 → 9000 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3837335662 TSecr=0 SACK_PERM
5413	89.268481	92.51.21.171	192.168.1.8	TCP	74	9000 → 54492 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=1094814817 TSecr=3837335662 WS=128
5414	89.268694	192.168.1.8	92.51.21.171	TCP	66	54492 → 9000 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=3837335723 TSecr=1094814817
5415	89.268770	192.168.1.8	92.51.21.171	TCP	380	54492 → 9000 [PSH, ACK] Seq=1 Ack=1 Win=131776 Len=314 TSval=3837335723 TSecr=1094814817
5424	89.324867	92.51.21.171	192.168.1.8	TCP	66	9000 → 54492 [ACK] Seq=1 Ack=315 Win=64896 Len=0 TSval=1094814873 TSecr=3837335723
5425	89.327013	92.51.21.171	192.168.1.8	TCP	168	9000 → 54492 [PSH, ACK] Seq=1 Ack=315 Win=64896 Len=102 TSval=1094814875 TSecr=3837335723
5426	89.327220	192.168.1.8	92.51.21.171	TCP	66	54492 → 9000 [ACK] Seq=315 Ack=103 Win=131712 Len=0 TSval=3837335782 TSecr=1094814875
5427	89.327389	192.168.1.8	92.51.21.171	TCP	66	54492 → 9000 [FIN, ACK] Seq=315 Ack=103 Win=131712 Len=0 TSval=3837335782 TSecr=1094814875
5428	89.327602	192.168.1.8	92.51.21.171	TCP	78	54502 → 9000 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=237101481 TSecr=0 SACK_PERM
5429	89.327607	192.168.1.8	92.51.21.171	TCP	78	54504 → 9000 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3221514196 TSecr=0 SACK_PERM
5430	89.327609	192.168.1.8	92.51.21.171	TCP	78	54503 → 9000 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=1638133100 TSecr=0 SACK_PERM
5437	89.383267	92.51.21.171	192.168.1.8	TCP	74	9000 → 54502 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=1094814933 TSecr=237101481 WS=128
5438	89.383268	92.51.21.171	192.168.1.8	TCP	66	9000 → 54492 [FIN, ACK] Seq=103 Ack=316 Win=64896 Len=0 TSval=1094814932 TSecr=3837335782
5439	89.383583	192.168.1.8	92.51.21.171	TCP	66	54502 → 9000 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=237101537 TSecr=1094814933
5440	89.383652	192.168.1.8	92.51.21.171	TCP	223	54502 → 9000 [PSH, ACK] Seq=1 Ack=1 Win=131776 Len=157 TSval=237101537 TSecr=1094814933
5441	89.383770	192.168.1.8	92.51.21.171	TCP	66	54492 → 9000 [ACK] Seq=316 Ack=104 Win=131712 Len=0 TSval=3837335838 TSecr=1094814932
5442	89.383820	92.51.21.171	192.168.1.8	TCP	74	9000 → 54504 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=1094814934 TSecr=3221514196 WS=128
5443	89.383961	192.168.1.8	92.51.21.171	TCP	66	54504 → 9000 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=3221514252 TSecr=1094814934
5444	89.383993	192.168.1.8	92.51.21.171	TCP	223	54504 → 9000 [PSH, ACK] Seq=1 Ack=1 Win=131776 Len=157 TSval=3221514252 TSecr=1094814934
5445	89.386067	92.51.21.171	192.168.1.8	TCP	74	9000 → 54503 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=1094814934 TSecr=1638133100 WS=128
5446	89.386207	192.168.1.8	92.51.21.171	TCP	66	54503 → 9000 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=1638133150 TSecr=1094814934

На скриншоте Wireshark показан обмен TCP-пакетами между клиентом 192.168.1.8 и сервером 92.51.21.171:9000. Видно несколько TCP-соединений к порту 9000: 54492 -> 9000, 54502 -> 9000, 54503 -> 9000, 54504 -> 9000. Это подтверждает параллельную передачу файлов несколькими соединениями.

Также на скриншоте видны параметры TCP handshake:

- MSS = 1460.
- У клиента в SYN: Win = 65535, WS = 64.
- У сервера в SYN/ACK: Win = 65160, WS = 128.
- SACK permitted включен.
- TCP timestamps включены.

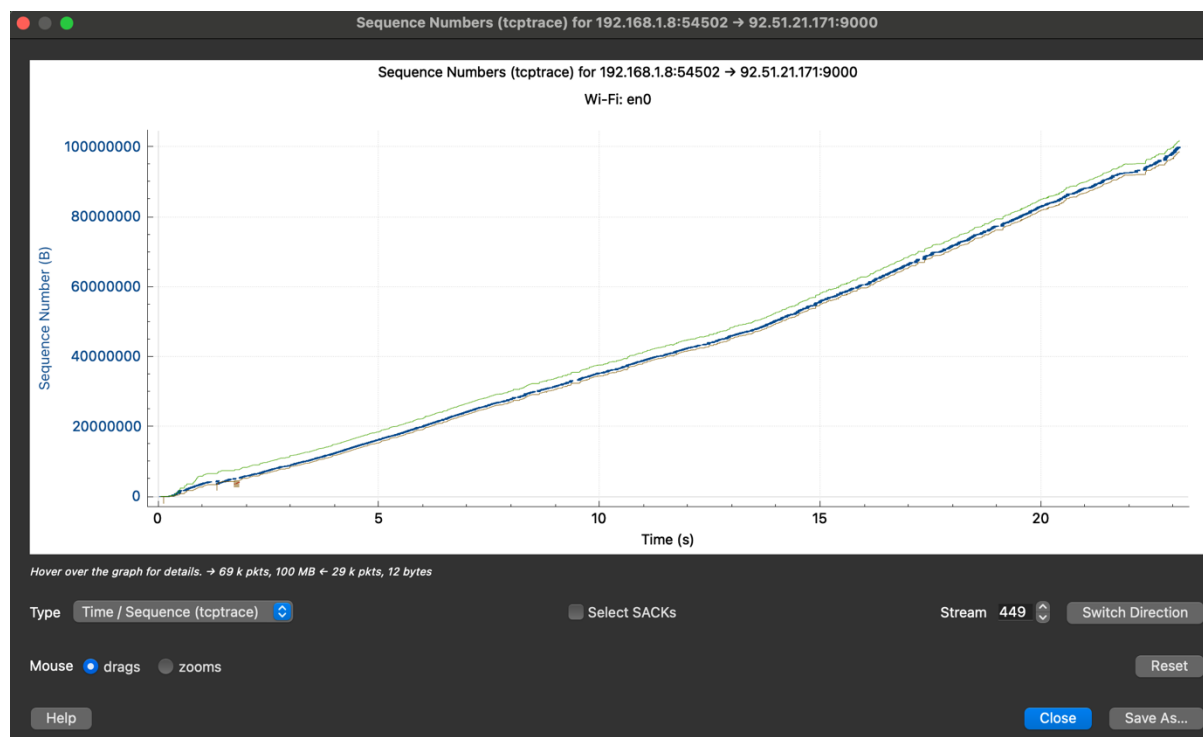
Расчетное окно приема:

- Клиент: $65535 * 64 = 4\,194\,240$ байт.
- Сервер: $65160 * 128 = 8\,340\,480$ байт.

На скриншоте также присутствует один RST/ACK для соединения 9000 -> 52076.

Этот пакет относится к отдельному закрытому соединению и не относится к

успешной передаче файлов, которая показана ниже на графике



На графике Wireshark Time / Sequence (tcptrace) для TCP stream 449 показана

передача данных от клиента 192.168.1.8:54502 к серверу 92.51.21.171:9000.

График sequence number растёт почти линейно от 0 до примерно 100 000 000 байт,

что соответствует передаче файла размером около 100 MB. В нижней части окна

Wireshark видно: 69 k packets, 100 MB.

No.	Time	Source	Destination	Protocol	Length	Info
200.	90.887342	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4552670 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887341	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4551222 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887339	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4549774 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887338	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4548326 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887337	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4546878 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887336	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4545430 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887335	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4543982 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887333	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4542534 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887332	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4541086 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887329	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4539638 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887326	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4538190 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
200.	90.887318	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4536742 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
198.	90.858567	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4535294 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
198.	90.858566	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4533846 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
198.	90.858564	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4532398 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
198.	90.858563	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4530950 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
198.	90.858562	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4529502 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
198.	90.858561	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4528054 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095
198.	90.858559	192.168.1.8	92.51.21.171	TCP	1514	[TCP Retransmission] 54502 → 9000 [ACK] Seq=4526606 Ack=13 Win=131776 Len=1448 TSval=237102833 TSecr=1094816095

На скриншоте применен фильтр Wireshark

tcp.stream == 449 && tcp.analysis.retransmission. Фильтр показывает повторные передачи TCP-сегментов от клиента 192.168.1.8:54502 к серверу 92.51.21.171:9000.

В строках видны сегменты с Len = 1448 байт и пометкой [TCP Retransmission].

Это означает, что во время передачи файла в stream 449 часть TCP-сегментов была передана повторно.

Наблюдения по TCP

1. MSS

В SYN-пакетах Wireshark показывает MSS = 1460. Это типичное значение для Ethernet MTU 1500, так как 1500 - 20 байт IP-заголовок - 20 байт TCP-заголовок = 1460 байт полезной нагрузки TCP-сегмента.

2. Window и Window Scaling

На скриншоте handshake видно:

- клиент отправляет SYN с Win = 65535 и WS = 64;
- сервер отвечает SYN/ACK с Win = 65160 и WS = 128.

Следовательно, обе стороны используют TCP Window Scaling, что важно для передачи больших файлов через сеть с заметной задержкой.

3. Slow start и изменение скорости

По графику Time / Sequence (tcptrace) нельзя строго доказать slow start, так как график в целом выглядит почти линейным. В начале передачи заметен короткий участок разгона, однако без дополнительного увеличения масштаба первых секунд и анализа congestion window корректнее сделать осторожный вывод: явный slow start на данном скриншоте не выделяется, передача быстро вышла на устойчивую скорость. Небольшие изменения наклона графика показывают адаптацию скорости TCP к условиям сети.

4. Congestion и retransmissions

Для stream 449 были обнаружены TCP retransmissions. Это видно по фильтру Wireshark:

```
tcp.stream == 449 && tcp.analysis.retransmission
```

Wireshark показывает повторные передачи сегментов от клиента к серверу: 192.168.1.8:54502 -> 92.51.21.171:9000. Размер TCP payload в показанных сегментах составляет Len = 1448 байт. Наличие retransmissions означает, что в ходе передачи часть сегментов или подтверждений была потеряна либо не была получена вовремя, после чего TCP повторно отправил соответствующие данные.

На графике Time / Sequence это проявляется как небольшие ступеньки и изменения наклона линии, но без длительной остановки передачи. TCP смягчает такие эффекты за счет повторной передачи потерянных сегментов, подтверждений ACK, управления окном и congestion control. В результате передача файла продолжилась и была успешно завершена.

Вывод

В результате работы было реализовано TCP-приложение MyDrive на C++20 с использованием Boost.Asio без coroutines. Клиент отправляет серверу manifest локальной директории, сервер определяет отсутствующие или измененные файлы, а клиент передает только необходимые файлы. Реализованы параллельная передача файлов через несколько TCP-соединений и два режима передачи: buffered и sendfile/DMA zero-copy.

Эксперименты подтвердили работоспособность приложения на удаленном сервере 92.51.21.171:9000. Были переданы файлы размером 50 MB, 150 MB, 200 MB и крупный файл размером 1500 MB. Wireshark показал несколько TCP-соединений, параметры MSS и Window Scaling, а график Time / Sequence подтвердил передачу файла около 100 MB по TCP.

Выполненные критерии на 10 баллов

Базовая оценка, до 5/6 баллов: выполнено.

Реализованы TCP client-server, client_id, manifest файлов, checksum, отправка недостающих или измененных файлов и хранение файлов на сервере отдельно по пользователю.

Параллельная отправка нескольких файлов, до +2 баллов: выполнено.

Клиент открывает несколько TCP-соединений и отправляет файлы параллельно. Поддерживается настройка connections от 1 до 32.

Передача файлов с DMA / zero-copy, до +2 баллов: выполнено.

Реализован режим sendfile, а также режим buffered для сравнения. Замеры buffered и sendfile приведены в отчете.

C++ / Boost.Asio без coroutines, максимум до 10 баллов: выполнено.

Проект написан на C++20 с Boost.Asio, coroutines не используются.

Итого: проект претендует на оценку до 10 баллов.